

构建运行与实验任务：从代码到提交

目 录

1 引言：把优化落地到提交	2
2 接口与计算语义	2
3 代码框架	2
4 选择开发路径	3
5 修改范围与限制	4
6 构建与自测	4
7 如何获取计算资源	4
7.1 注意区分 Devpod	5
8 评分方式	5
8.1 正确性验证	5
8.2 性能评分	5
9 实验报告要求	6
10 思考题	6
11 提交要求	6
12 本章你将学会	7
13 要点速查	7
14 小结	7

1 引言：把优化落地到提交

前面几章讲了算子、硬件、编程模型、profiling 和优化策略，本章把这些知识落地到实际操作：代码框架长什么样，怎么选择开发路径，怎么构建和自测，怎么获取计算资源，评分标准是什么，最后要交什么。读完本章你就能开始动手做实验了。

直觉 不妨把前几章想成学开车（认车、学交规、练车技），本章是“考驾照”的流程：在哪考、怎么报名、考什么、怎么算过、最后拿什么证。

本章我们先看算子的接口与计算语义，再梳理代码框架与开发路径，然后讲构建与自测命令，最后说明计算资源、评分与提交要求。

2 接口与计算语义

设 B 为行数， H 为隐藏层宽度，算子的输入输出如下：

张量	参数类型	形状	数据类型	含义
x	输入	$[B, H]$	FP16	输入
$residual$	输入	$[B, H]$	FP16	残差
$weight$	输入	$[H]$	FP16	RMSNorm 缩放权重
eps	输入	标量	FP16	防止除零的微小偏移
y	输出	$[B, H]$	FP16	归一化结果
$residual_out$	输出	$[B, H]$	FP16	残差加法结果

对每一行 $b \in [0, B)$ ，算子计算：

$$R_b = x_b + residual_b \quad (2.1)$$

$$rms_b = \sqrt{\frac{1}{H} \sum_{i=0}^{H-1} R_{b,i}^2 + \varepsilon} \quad (2.2)$$

$$y_b = \frac{R_b}{rms_b} \cdot w \quad (2.3)$$

$$residual_out_b = R_b \quad (2.4)$$

三个开发路径对 checker 暴露的入口均为 `fused_add_rmsnorm(x, residual, weight, eps)`，其中 `eps` 默认为 10^{-6} 。

本实验采用非原地接口： x 和 $residual$ 为只读输入，返回独立的 y 与 $residual_out$ 。请勿改变这一约定。

3 代码框架

实验代码位于仓库的 `src/lab3p5/`：

```
src/lab3p5/
├─ env.sh                # 加载课程 CANN 与 Python 环境
├─ README.md            # 代码框架使用与提交说明
└─ checker/
```

```

|   |─ build.sh                # 构建并安装 Ascend C 算子
|   |─ run.sh                  # 正确性检查
|   |─ profile.sh              # 固定性能 case 的 msprof 采集
|   |─ test_op.py              # 输入生成, FP32 golden 与逐元素比较
|   |─ case_specs.py           # 公开测试配置
|   |─ get_time.py             # 解析 op_summary 中的 kernel 时间
└─ src/
    |─ __init__.py
    |─ ascendc/
    |   |─ op_host/
    |   |   |─ CMakeLists.txt
    |   |   |   └─ fused_add_rms_norm.cpp      # 算子注册与 Host tiling
    |   |─ op_kernel/
    |   |   |─ CMakeLists.txt
    |   |   |   └─ fused_add_rms_norm.cpp      # Device kernel
    |   |   |   └─ fused_add_rms_norm_tiling.h # Tiling 数据结构
    |   |─ extension/custom_op.cpp            # PyTorch 扩展胶水
    |   |─ common/pytorch_npu_helper.hpp      # PyTorch NPU 辅助头文件
    |   |─ CMakeLists.txt / CMakePresets.json
    |   |─ build_op.sh                         # 构建 Ascend C 算子与 wheel
    |   |─ setup.py
    |─ triton/
    |   |─ __init__.py
    |   |─ fused_add_rmsnorm.py                # Triton kernel 与 launcher
    └─ tilelang/
        |─ __init__.py
        |─ fused_add_rmsnorm.py                # TileLang kernel 与 launcher

```

4 选择开发路径

路径	建议先阅读	运行方式
Ascend C	src/ascendc/op_host/ fused_add_rms_norm.cpp, src/ ascendc/op_kernel/ fused_add_rms_norm.cpp	直接运行 checker, 脚本按需构建并 安装算子
Triton-Ascend	src/triton/fused_add_rmsnorm.py	提交任务时设置环境变量 LANG=triton
TileLang-Ascend	src/tilelang/ fused_add_rmsnorm.py	提交任务时设置环境变量 LANG=tilelang

Ascend C 路径提供了一个以正确性为主的 baseline: Host 侧读取 shape 和属性并生成 tiling; Device 侧按行分配工作, FP16 数据进入 UB 后主要使用 FP32 计算, 再转换为 FP16 输出。它已经展示了 TQue、TBuf、数据搬运、Vector 计算和规约的基本组织方式, 但同步、流水和 tiling 仍有优化空间。

你只需要选择一种路径实现即可, 多种实现不会带来进一步加分。课程鼓励使用 Ascend C, 但使用 Ascend C 路径不会自动带来额外加分。

5 修改范围与限制

你可以修改所选路径目录下的实现，并在该目录内增加必要的辅助文件：

1. Ascend C：可修改 `src/ascendc/`
2. Triton-Ascend：可修改 `src/triton/`
3. TileLang-Ascend：可修改 `src/tilelang/`

允许自行设计 kernel、tiling、核数、片上存储布局和针对不同 shape 的实现分支。**禁止：**

1. 修改 checker/、env.sh、输入生成、golden 或计时逻辑。
2. 调用已有 RMSNorm、FusedAddRmsNorm 或等价高层算子代替被测计算。
3. 硬编码测试数据、隐藏 shape 或输出结果。
4. 利用评测程序漏洞绕过计算或正确性检查。
5. 依赖课程环境中未提供的额外软件包或自建工具链。

可以参考开源实现，但需要在报告中注明来源，并说明自己的实现与修改。

6 构建与自测

所有命令都应在 `src/lab3p5/` 下执行。NPU 任务通过 `hpc submit` 提交到 lab3p5 分区；`run.sh` 和 `profile.sh` 会自行加载 `env.sh`，一般不需要在提交任务前手动 `source`。

正确性：运行全部公开 case

```
hpc submit -p lab3p5 bash checker/run.sh
hpc submit -p lab3p5 -e LANG=triton bash checker/run.sh
hpc submit -p lab3p5 -e LANG=tilelang bash checker/run.sh
```

正确性：只运行一个公开 case；编号从 1 开始

```
hpc submit -p lab3p5 bash checker/run.sh 2
```

性能：固定 shape 性能测试 (256×1024)，不接受 case 参数

```
hpc submit -p lab3p5 bash checker/profile.sh
hpc submit -p lab3p5 -e LANG=triton bash checker/profile.sh
hpc submit -p lab3p5 -e LANG=tilelang bash checker/profile.sh
```

`checker/run.sh` 只负责正确性检查；`checker/profile.sh` 只采集 student 算子的性能，使用 `msprof op --warm-up=10` 并输出一次 Task Duration(us)。进行性能测试前，应先用 `run.sh` 验证正确性。

Ascend C 路径下，每次改动代码后需要自己进行编译，编译脚本已经写好为 `checker/build.sh`，在 Devpod 内即可进行。TileLang 和 Triton 路径会自动触发编译，不需要手动进行。

7 如何获取计算资源

通过实验平台提供 **arm64-910b** DevPod，容器拉取的镜像中已经配有本实验所需的 CANN 工具链和 Triton/Tilelang 包环境，一般不需要自行安装工具链。你需要做的包括：

1. 登录实验平台。

2. 创建预设为 arm-910b 的 DevPod。
3. 在 DevPod 中获取课程仓库并进入 src/lab3p5/。
4. 执行 `source ./env.sh` 后开始构建和测试。
5. 在需要在 NPU 上执行算子时使用 `hpc submit -p lab3p5 <your commands>` 即可提交至分配有一张 910B4 NPU 的计算分区执行你的命令。

家目录不共享。华为提供的 Ascend 910B4 8 卡裸金属机器地理上分布于华北-乌兰察布地区，距离超算队在杭州的集群和其他硬件资源有一定地理距离。由于 NFS 对于时延有较高要求，本平台的家目录不会与其他硬件资源的家目录共享。

7.1 注意区分 Devpod

在创建 Devpod 时，Lab 4 任务一所需的鲲鹏环境对应的 Devpod 预设为 arm64-920b，而 Lab3.5 的预设 Devpod 为 arm64-910b，两者环境和家目录均不互通，**请注意区分**。

8 评分方式

评测包括**正确性**和**性能**两部分。只有通过正确性检查的实现才会进入性能计分。

8.1 正确性验证

我们提供多个 Case 测试你的算子。公开 case 由 checker/case_specs.py 定义：

case 索引	命令行编号	$B \times H$	说明
0	1	32×4096	小规模, 对齐
1	2	256×1024	性能评测配置
2	3	1×4096	单行
3	4	1997×3037	行数与尾部均不对齐
4	5	2048×4096	大规模, 对齐

输入由固定 seed 在运行时**随机**生成。正式评测还会使用不同 shape 的隐藏 case（保证数据范围大致一致），因此不能只针对公开配置硬编码实现。

Golden 在 FP32 下完成残差加法、平方和、均值、开方、除法和权重缩放，最后转换为 FP16。y 与 residual_out 均需逐元素通过检查。对参考值 g_i 和输出值 o_i ，元素在满足下列任一条件时通过：

$$|o_i - g_i| \leq 10^{-3} \quad \text{或} \quad \frac{|o_i - g_i|}{\max(|g_i|, 10^{-12})} \leq 10^{-3} \quad (8.1)$$

整个张量要求错误元素比例为 0。一个更快但未通过全部正确性检查的实现不会获得性能分数。

8.2 性能评分

性能测试只涉及单个 Shape，评测配置为：Shape [256, 1024]，输入输出类型 FP16， $\text{eps} = 10^{-6}$ ，指标为被测 kernel 的 Task Duration(us)（来自 msprof op 的结果，热身 10 次）。

Ascend C 基线性能为起始评分点，满分 120 分，超出的 20 分将作为 Bonus。

9 实验报告要求

实验报告提交 PDF，重点说明你如何从测量得到优化决策，不需要重复大段背景知识或 API 文档。报告至少应包含：

1. 使用的开发路径、测试环境、软件版本和运行命令。
2. 算子的计算过程、数据依赖和初始实现。
3. baseline 的正确性、性能数据和 profiling 证据。
4. 每项主要优化针对的瓶颈、关键修改及其收益。
5. 最终正确性结果和性能结果。
6. 尝试过但未采用的方案及原因。
7. 思考题作答。
8. 参考过的资料或开源实现。

关于 AI 使用：本实验允许使用 *AI Agent* 辅助开发和理解资料，但最终报告应由你自行组织和核实，禁止使用 AI 生成。

10 思考题

部分思考题可能没有一个标准的正确答案，更希望看到你在实验过程中遇到的实际情况以及你个人的思考和理解。

1. 对 [256, 1024]、FP16 配置，估算算子必须进行的 GM 读写量和主要浮点操作数。说明你的计数口径，计算算术强度，并结合 msprof 结果判断实现更接近计算瓶颈还是访存瓶颈。
2. 你的算子是否开启了 Double Buffer 流水？请通过 msprof op simulator 的结果证明。如果开启了双缓冲流水，说明你是显式编写了依赖还是让编译器自动识别开启的？
3. (Bonus) TQue 本身是否是一个真实存在的队列？我们在 EnQue 和 DeQue 时相关的 LocalTensor 是否发生了在队列之间的拷贝或移动？
4. (Bonus) 华为今年推出了新一代 NPU Ascend 950PR，相比 910 系列带来了哪些新的硬件特性？也可以比较昇腾 NPU（910B 系列）与 NVIDIA GPU 在硬件设计理念、开发语言等方面的异同。
5. (Bonus) 分享你对昇腾 NPU 的使用体验或相关开发语言（Ascend C、Triton-ascend、TileLang-ascend）的体验。

11 提交要求

需要提交：

1. 实现代码：所选开发路径对应的完整目录。
2. 实验报告：单独的 PDF 文件。

开发路径	上传目录	平台放置位置
Ascend C	src/ascendc/	src/ascendc/
Triton-Ascend	src/triton/	src/triton/
TileLang-Ascend	src/tilelang/	src/tilelang/

新增辅助文件必须位于所选路径目录内，并保证代码能在课程提供的干净环境中构建和运行。请勿提交 checker/、env.sh 或对这些文件的修改，以及 build_out/、dist/、wheel、custom OPP 安装目录等构建产物。

12 本章你将学会

1. 描述 FusedAddRmsNorm 的接口（输入 x/residual/weight/eps，输出 y/residual_out）和计算语义。
2. 读懂 src/lab3p5/ 的代码框架，区分 checker/ 与 src/ascendc|triton|tilelang/。
3. 用 hpc submit -p lab3p5 提交正确性检查（run.sh）和性能采集（profile.sh）。
4. 创建 arm-910b DevPod，区分它与 Lab4 的 arm64-920b 预设。
5. 说明正确性阈值（相对或绝对 $\leq 10^{-3}$ ）和性能指标（Task Duration(us)，热身 10 次）。
6. 列出实验报告应包含的内容，以及禁止提交的文件。

13 要点速查

概念	要点
代码位置	src/lab3p5/, 含 checker/ 和 src/ascendc triton tilelang
接口	fused_add_rmsnorm(x, residual, weight, eps), eps 默认 10^{-6}
输入输出	FP16, 非原地接口
正确性命令	hpc submit -p lab3p5 bash checker/run.sh [N]
性能命令	hpc submit -p lab3p5 bash checker/profile.sh
性能 shape	[256, 1024], Task Duration(us), 热身 10 次
正确性阈值	相对或绝对误差 $\leq 10^{-3}$, 错误比例 0
DevPOD	预设 arm-910b, 与 Lab4 的 arm64-920b 区分
提交	所选路径目录 + 报告 PDF
禁止修改	checker/, env.sh, golden, 计时逻辑
Bonus	满分 120, 超出 20 分为 Bonus

14 小结

本章把前几章的知识落地到实际操作：我们从算子的接口与计算语义出发，梳理了 src/lab3p5/ 的代码框架与三种开发路径的构建运行方式，说明了 arm-910b DevPOD 的获取与 hpc submit 提交流程。评分上，先过正确性（误差 $\leq 10^{-3}$ ）才能进入性能计分，性能以 [256, 1024] 下的 Task Duration 为指标。最后我们明确了实验报告应包含的内容、思考题与提交要求。至此你已经具备了完成 Lab3.5 的全部知识储备，可以开始动手优化了。

讲义基于 HPC101 2025 Lab3.5 实验内容编写